

IoT Routine Detection and Urban Mobility Pattern Recognition

Alessia Grandetti, Nicola Lerco, and Sara Ventura

University of Camerino, School of Science and Technologies
{alessia.grandetti, nicola.lerco, sara.ventura}@studenti.unicam.it

Abstract. This paper extends the routine detection framework proposed by De Donato et al., which combines community detection on Directly-Follows Graphs with Complex Event Processing to identify behavioural patterns in IoT event streams. We implement the full pipeline on Apache Flink, defining eight detection strategies that differ in the number of required events, contiguity constraints, and the presence of a temporal window, and evaluate them on a real-world smart-home sensor dataset covering the daily activities of a single resident. The results confirm that the approach successfully detects human routines: exact-count strategies identify compact, well-defined behavioural sequences, while percentage-based strategies substantially improve recall by tolerating partial occurrences of a routine, at the cost of a modest reduction in precision. As an exploratory extension, we apply the same architecture without modification to the NYC Yellow Taxi trip records, treating borough-level pickup areas as communities. This second scenario reveals that the effectiveness of the approach is critically dependent on community granularity: with communities containing up to 69 zones, most strategies fail to produce meaningful detections, and only the most permissive strategy yields consistent results. These findings suggest that the two-phase architecture generalises across domains, but that community definition — rather than strategy selection — is the key design decision when applying it to new contexts.

Chapter 2

Background

This chapter introduces the context from which the project originates and the key technologies on which it is built. Readers already familiar with Apache Flink and Complex Event Processing may skip to Chapter 3.

1 Project Context

This project extends the work presented in Callisto De Donato. [2], which proposes a two-phase approach to detect human routines from smart-home IoT event logs. The first phase applies community detection algorithms to a Directly-Follows Graph built from historical data, grouping events that tend to co-occur into communities. The second phase uses Apache Flink CEP to detect occurrences of those communities in a real-time event stream.

Starting from that foundation, we continued the implementation of the smart-home scenario and then extended the same architecture to a second, substantially different domain: the NYC Yellow Taxi trip records. The two scenarios share the same processing pipeline and the same eight detection strategies, but differ in how events are structured, how communities are defined, and how individual traces are identified. These differences are described in detail in Chapter 4.

2 Apache Flink

Apache Flink is an open-source distributed stream processing framework designed to process unbounded and bounded data streams in real time [1]. In Flink, a computation is expressed as a directed acyclic graph of operators, where each operator receives a typed input stream and emits a typed output stream. Operators are connected by the framework and executed in parallel across a cluster of worker nodes.

Flink supports two notions of time. Processing time is the wall-clock time at which an event is processed by an operator. Event time is the timestamp embedded in the event itself, which may differ from processing time if events arrive out of order or with a delay. This project uses event time throughout, because the correctness of CEP pattern detection depends on the chronological order of events, not on when they happen to arrive at the system.

To reason about event time, Flink uses **watermarks**: special records injected into the stream that signal that all events up to a given timestamp have been observed. When a watermark arrives at an operator, it can safely process all buffered events with timestamps below that watermark in chronological order. In this project, both input files are already sorted by timestamp, so the watermark strategy `forMonotonousTimestamps()` is used. It advances the watermark to the timestamp of the last seen event, which means there is no buffering and events are processed as they arrive.

3 Complex Event Processing with FlinkCEP

FlinkCEP is the Complex Event Processing library built on top of Apache Flink. As stated in the official documentation, it allows detecting event patterns in an endless stream of events, giving the opportunity to identify what is important in the data [1].

The central concept is the **pattern**: a declarative specification of a sequence or combination of events, together with optional constraints on contiguity, cardinality, and timing, that must be satisfied for the engine to declare a match. Patterns are constructed using the **Pattern** fluent API and then applied

to a `KeyedStream` via `CEP.pattern()`, which returns a `PatternStream`. Each match is processed by a `PatternProcessFunction` that receives the matched events and can emit any number of output records.

Internally, FlinkCEP compiles each pattern into a **Non-deterministic Finite Automaton** (NFA) that is maintained per key in the keyed stream [1]. One NFA instance exists for each distinct key, meaning that all cases are analysed independently and in parallel. As events arrive, the NFA transitions between states according to the pattern conditions. When the NFA reaches an accepting state, a match is emitted.

3.1 Contiguity Modes

A central design choice in any FlinkCEP pattern is the **contiguity mode**, which determines whether non-matching events are allowed to appear between two consecutively matched events. FlinkCEP provides three modes [1].

Strict contiguity (`next()` or `consecutive()`) requires all matching events to appear strictly one after the other, with no non-matching events in between. If a non-matching event arrives while the NFA is waiting for the next pattern step, the partial match is discarded.

Relaxed contiguity (`followedBy()`) ignores non-matching events between two matched events. The engine skips over events that do not satisfy the current condition and waits for the next one that does.

Non-deterministic relaxed contiguity (`followedByAny()`, or `allowCombinations()` on looping patterns) further relaxes this by exploring all possible combinations of matching events, which can produce multiple overlapping matches for the same input sequence.

When applied to looping quantifiers such as `times(n)` or `oneOrMore()`, strict contiguity is enforced via `consecutive()` and non-deterministic relaxed contiguity via `allowCombinations()`.

3.2 Temporal Constraints and After-Match Skip

FlinkCEP allows defining a maximum time interval within which a pattern must be completed via the `within()` method [1]. If the pattern is not completed within the specified duration, the partial match is discarded and the NFA state is reset. The time window interacts directly with the watermark strategy: the engine checks for expired partial matches whenever a watermark advances.

An equally important parameter is the **after-match skip strategy**, which controls which events can still participate in subsequent matches after a match has been found. All eight strategies in this project use `AfterMatchSkipStrategy.skipPastLastEvent()`, which discards all events up to and including the last event of the current match before looking for the next one. This prevents the engine from producing heavily overlapping detections of the same sequence of events.

Chapter 3

Community, Event, and Strategy

This chapter defines the three core concepts of the system and explains how they relate to each other. Understanding these concepts is a prerequisite for reading the architecture and results chapters.

4 Event

An **event** is a single atomic action recorded at a specific point in time. It is the smallest unit of data that the system processes. In the smart-home scenario, an event is an IoT sensor activation, for example `go_computer` or `switch_computer_on`. In the taxi scenario, an event is a single taxi pickup recorded at a specific zone, for example a pickup at zone 161 (Midtown Center).

Every event carries at minimum a timestamp, an activity label, and a case identifier. The timestamp establishes when the event occurred. The activity label describes what the entity was doing. The case identifier groups related events into a trace, which is the sequence of events produced by one entity over one unit of time.

5 Design Decision

During development, several design decisions were made that deserve explicit justification.

Watermark strategy. The choice of `forMonotonousTimestamps()` over `forBoundedOutOfOrderness(Duration.ofSeconds(5))` was made because both input files are sorted by timestamp. Using a bounded out-of-orderness tolerance would have added a 5-second processing delay after the last event (the watermark would not advance past the last timestamp until 5 seconds of “wall-clock silence” had elapsed). Since the input is a static file and not a live stream, this delay served no purpose. For a live deployment, the tolerance value would need to be estimated from empirical latency measurements rather than set arbitrarily.

Hardcoded case identifier. The smart-home pipeline uses `"house_01"` as the hardcoded `caseId`. This is appropriate for the single-house, single-resident dataset in use, but it is a known limitation: in a multi-home deployment, all events would share the same trace and the NFA would produce spurious matches across different houses. The taxi pipeline generalises this by constructing the case identifier dynamically as `vendorId + "_" + date`.

Separate output files per community. The pipeline writes one output file per community rather than one combined file. This design choice makes it easier to inspect the results for a specific community without parsing a large combined file, and it allows different communities to be processed by different downstream tools independently. A combined file would be more convenient for cross-community analysis and could be produced as a post-processing step by concatenating the per-community files.

Handling duplicates. The `AfterMatchSkipStrategy.skipPastLastEvent()` skip strategy reduces (but does not eliminate) duplicate detections. A match is considered a duplicate if it covers the same set of events as a previous match. With `skipPastLastEvent()`, the NFA discards all events up to and including the last matched event before searching for the next match, which prevents immediately consecutive overlapping matches. However, Strategy C (`allowCombinations()`) can still produce multiple matches for the same input subsequence by exploring different subsets of the matching events. Deduplication of the output (by comparing matched event sets) would need to be implemented as a post-processing step.

Checkpoints. Flink checkpoints were not enabled in this implementation. Checkpoints are used to recover from failures in long-running streaming jobs by periodically saving the NFA state to durable storage. Since both pipelines read from static files and complete within minutes, failure recovery is not a concern and checkpoints would only add overhead. For a live streaming deployment reading from Kafka, checkpoints would be essential.

Parallelism. The job is configured with parallelism 1. Higher parallelism would distribute the processing of different keys (houses, vendor-day pairs) across multiple task slots, reducing latency. However, it would also require that the file source and sinks are partitioned-friendly, which the current single-file input and single-directory output already support through Flink’s partitioned file infrastructure. For the current scale of input (one CSV file), parallelism 1 is adequate.

5.1 Vendor Filtering

The `--vendor` argument allows running the pipeline on a single vendor’s data. Its rationale is the following. In the taxi dataset, only two vendors are present (`VendorID` 1 and 2). Without filtering, both vendors’ events are mixed in the same input stream. The case identifier (`vendorId.date`) correctly partitions them into separate traces, so the CEP engine analyses the two vendors independently and correctly. However, the output files contain detections from both vendors interleaved, making comparison harder.

Running the pipeline twice, once with `--vendor 1` and once with `--vendor 2`, produces two separate sets of output files that can be compared directly. Vendor 1 and Vendor 2 may exhibit different borough preferences: one may operate predominantly in Manhattan, the other in Queens or Brooklyn. These differences are visible in the detected routine counts per community per strategy.

The filtering is implemented inside `CSVInputParser`: when `vendorFilter` is non-null, any line whose `VendorID` field does not match is returned as `null` and dropped by the downstream null filter. This is more efficient than filtering after parsing, because it avoids constructing `Event` objects for trips that will immediately be discarded.

6 Detection Strategy

A **detection strategy** is the rule that defines how many events from a community must be observed, in what order, and within what time window, for the system to declare that the corresponding routine or routing pattern has occurred.

Eight strategies are available, each offering a different trade-off between strictness and recall. They differ along three dimensions: the number of events required, the contiguity mode (strict, relaxed, or non-deterministic relaxed), and whether a time window is enforced.

Let N denote the number of events in a community and $p \in (0, 1]$ the percentage parameter, configurable via the `--percent` CLI argument with a default of 0.8. All strategies use `AfterMatchSkipStrategy.skipPastLastEvent()` to prevent overlapping matches.

It is important to note that the eight strategies are defined at a conceptual level and are shared between the two implementations. However, the concrete realisation of several strategies differs between the smart-home and the taxi scenario, reflecting the structural differences between the two domains. These differences are described in detail in Chapter 4. The descriptions below capture the intended behaviour of each strategy; where the two implementations diverge, the divergence is noted explicitly.

Strategy A requires all N events of a community to appear in the stream, tolerating interleaving events from other communities between matched steps. In the smart-home implementation, it is realised as a chain of `followedByAny()` steps that check both community membership and exact activity identity, with no time window. In the taxi implementation, it uses a single `.times(N)` quantifier that checks only community membership, with no time window.

Strategy B requires all N events of a community to appear in the exact order defined by the community list, with relaxed contiguity (`followedBy()`) and no time window. It checks exact activity identity at

each step. In the smart-home implementation each step also verifies community membership; in the taxi implementation it checks only the activity name. This is the only strategy that enforces a specific activity ordering. Its practical effectiveness is severely limited in both scenarios because the community list order is determined by algorithmic artefacts rather than by the natural execution sequence of the routine.

Strategy C requires exactly N events from the community with non-deterministic relaxed contiguity, within the time window. It uses `.times(N).allowCombinations()`, which instructs the NFA to explore all possible subsets of matching events and may produce multiple overlapping matches for the same input subsequence.

Strategy D requires exactly N events from the community, strictly consecutive in the stream, within the time window. It uses `.times(N).consecutive()`, so no events from other communities may appear between the matched ones. It is the strictest of the exact-count strategies.

Strategy E relaxes the count requirement to at least $n = \lceil N \times p \rceil$ events from the community, with relaxed contiguity and a time window. In the smart-home implementation it uses `.timesOrMore(n)`, accepting any number of community events greater than or equal to n . In the taxi implementation it uses `.times(n)`, requiring exactly n events.

Strategy F is identical to Strategy E but adds `.allowCombinations()`, enabling non-deterministic relaxed contiguity. The NFA explores all possible subsets of at least n matching events and may produce multiple overlapping matches. In the smart-home implementation it uses `.timesOrMore(n).allowCombinations()`; in the taxi implementation `.times(n).allowCombinations()`.

Strategy G combines the percentage relaxation of E and F with strict contiguity. In the taxi implementation it uses `.times(n).consecutive()`, requiring exactly n strictly consecutive community events with no interleaving events from other communities. In the smart-home implementation it is realised as a chain of `followedBy()` steps of length n , checking both community membership and exact activity identity, mirroring the structure of Strategy B but limited to the first n activities in the list.

Strategy H is the most permissive strategy, but its implementation differs fundamentally between the two scenarios. In the taxi implementation it uses `.timesOrMore(2).consecutive().within(tMax)`, requiring at least two strictly consecutive community events within the time window. In the smart-home implementation it detects a temporal *gap* between two community events: it fires when a community event is followed by a silence of at least `tMax` with no further community events, followed by a new community event. The smart-home variant uses `notFollowedBy()` to express the absence of a community event within the window, and `followedBy()` to require a subsequent community event after the gap.

Table 1 and Table 2 summarise the eight strategies for the smart-home and taxi implementations respectively.

Table 1. Detection strategies – smart-home implementation.

Strategy	Count	Condition	Contiguity	Window
A	Exact N	Community + Activity	Non-det. relaxed (ordered)	No
B	Exact N	Community + Activity	Relaxed (ordered)	No
C	Exact N	Community	Non-det. relaxed	Yes
D	Exact N	Community	Strict	Yes
E	$\geq \lceil N \times p \rceil$	Community	None	Yes
F	$\geq \lceil N \times p \rceil$	Community	Non-det. relaxed	Yes
G	$\lceil N \times p \rceil$ first	Community + Activity	Relaxed (ordered)	No
H	Gap detection	Community	<code>notFollowedBy</code>	Yes

Table 2. Detection strategies – taxi implementation.

Strategy	Count	Condition	Contiguity	Window
A	Exact N	Community	None	No
B	Exact N	Activity	Relaxed (ordered)	No
C	Exact N	Community	Non-det. relaxed	Yes
D	Exact N	Community	Strict	Yes
E	Exact $\lfloor N \times p \rfloor$	Community	Non-det. relaxed	Yes
F	Exact $\lfloor N \times p \rfloor$	Community	None	Yes
G	Exact $\lfloor N \times p \rfloor$	Community	Strict	Yes
H	≥ 2	Community	Strict	Yes

6.1 Example of Detected Routines

To make the output of the system concrete, the following examples are taken from the actual output produced by running the system on the smart-home dataset with Strategy A.

The first example shows three detected instances of the `go_computer` community. Each row is one match. The matched activities vary across instances: the resident approached the computer in a different order each time, yet all three instances are correctly recognised as the same routine because Strategy A does not enforce any ordering constraint.

```
go_computer,A,house_01,2020-03-18T06:06:00Z,2020-03-18T06:07:00Z,
  go_computer;switch_computer_on;go_computer_chair
```

```
go_computer,A,house_01,2020-03-18T19:23:00Z,2020-03-18T19:24:00Z,
  go_computer;switch_computer_on;go_computer_chair
```

```
go_computer,A,house_01,2020-03-31T05:24:00Z,2020-03-31T05:29:00Z,
  go_computer_chair;use_the_computer;go_computer
```

The third instance shows a different ordering: the resident sat down before going to the computer. Both orderings are valid detections of the same community.

The second example shows instances of the `get_water` community, which models the Drink routine. The sequence varies across instances, with some including `pack_goods` or `put_plate_to_sink` as intermediate steps, yet all are recognised as the same routine.

```
get_water,A,house_01,2020-03-20T22:30:00Z,2020-03-20T22:40:00Z,
  go_kitchen_sink;go_kitchen_shelf;get_glass;get_water
```

```
get_water,A,house_01,2020-03-22T02:09:00Z,2020-03-22T02:17:00Z,
  go_kitchen_shelf;get_glass;get_water;drink_water
```

```
get_water,A,house_01,2020-03-26T07:07:00Z,2020-03-26T07:13:00Z,
  go_kitchen_shelf;pack_goods;get_glass;get_water
```

These examples illustrate the core value of the community-based approach: rather than requiring a fixed sequence of events, the system recognises a routine as long as enough events from the right community appear within the specified time window, tolerating the natural variability in human behaviour. Note that these examples are produced by the smart-home implementation of Strategy A, which uses `followedByAny()` steps with no time window; the taxi implementation of Strategy A uses `.times(N)` and would produce structurally different output.

Chapter 4

System Architecture

This chapter describes the technical implementation of the system. It covers the overall pipeline structure, the data model, the input parsing components, the enrichment phase, and the CEP detection phase. The description focuses on the smart-home scenario, which was the first implementation developed; the adaptations made for the NYC taxi scenario are discussed in Chapter 5.

7 Pipeline Overview

The system is structured as a single Apache Flink streaming job whose execution is controlled by a set of command-line arguments. At the top level, the `Main` class parses the arguments, loads the community definitions from disk, constructs the Flink dataflow graph, and submits the job. The graph is then executed by the Flink runtime, which processes the input file and writes the output.

The command-line interface accepts the following parameters.

- `--events`: path to the input CSV/TSV event log.
- `--communities`: path to the community definition file.
- `--out`: path to the output directory.
- `--strategy`: one of the eight strategy identifiers (A through H).
- `--mode`: either `enrichment` or `detection` (default: `detection`).

The system operates in two mutually exclusive modes. In *enrichment mode*, the pipeline reads the raw event log, maps each event to its community, and writes the enriched stream to disk. This mode is useful for inspecting the community assignment before running detection. In *detection mode*, the pipeline applies one of the eight CEP strategies to each community in turn and writes the detected routine instances.

The overall dataflow is shown below.

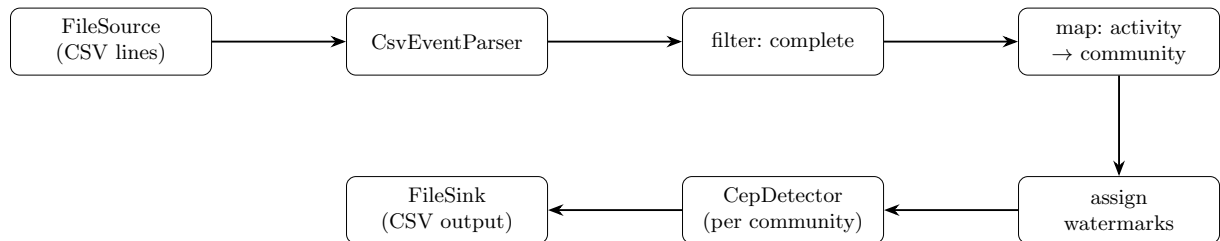


Fig. 1. Pipeline del job di elaborazione dati.

The job is configured with parallelism 1, which is appropriate for the single-file, single-house input and avoids any ordering issues that would arise when multiple parallel tasks process the same stream.

8 Data Model

Three immutable value types flow through the pipeline. Each type adds information to the previous one.

Event represents a raw record read from the input log. It carries a parsed **Instant** timestamp, the activity label as a string, a numeric **eventId**, a **caseId** string, and the **lifecycle** field extracted from the XES-style CSV (**lifecycle:transition**). Only events whose lifecycle is **"complete"** are retained; all others are discarded immediately after parsing.

EnrichedEvent extends a raw event by adding the **community** label. It is produced by the enrichment map operator and is the type consumed by the CEP engine. The community field is set to the result of the activity-to-community lookup; if the activity is not found in the mapping, the value **"unknown"** is used so that the event is not silently dropped but instead flows through the pipeline and is naturally excluded by every pattern condition.

DetectedRoutine is the output type of the CEP engine. It records the community name, the strategy letter, the case identifier, the start and end timestamps in milliseconds since epoch, and the ordered list of matched activity labels. It implements **Serializable** to satisfy Flink's type system requirements, exposes a full set of JavaBean getters and setters, and provides a **toCsvLine()** method that serialises the record to the comma-separated format.

9 Input Parsing

9.1 Event Log Parser

The **CsvEventParser** class implements Flink's **MapFunction<String, Event>** interface and is responsible for converting raw text lines from the file source into **Event** objects. It handles two design challenges that arise in practice: the header line and the ambiguity between tab-separated and comma-separated formats.

The parser is stateful. On the first non-empty line it receives, it detects the delimiter by counting the occurrences of **'\t'** and **','** in the line: whichever appears more frequently is chosen as the delimiter. It then parses the column names and records their positions in an index map (**idx**). On all subsequent lines, it uses the stored positions to extract the relevant fields. This approach makes the parser tolerant of any column ordering in the input file.

The mandatory columns are **time:timestamp** and **Activity**. The optional columns **eventId** and **lifecycle:transition** are read when present; if absent, default values (**-1** and **null** respectively) are used. The timestamp is parsed from ISO-8601 offset format after replacing any space separator with **'T'** so that both **2020-03-18 06:06:00+00:00** and **2020-03-18T06:06:00Z** are accepted. Lines that cannot be parsed are returned as **null** and filtered out by the downstream **filter(e -> e != null)** operator.

Activities whose name starts with the prefix **"noise"** (case-insensitive) are treated as noise events and removed by a dedicated filter operator. The noise removal is performed *after* the lifecycle filter and *before* the community enrichment, so that noise events never consume a community membership and never appear in the output.

9.2 Communities Parser

The **CommunitiesParser** class reads the community definition file from disk at startup, before the Flink job is submitted. The file format is one community per line, with a tab separating the community descriptor from the comma-separated list of activities:

```
go_computer:0    go_computer,switch_computer_on,go_computer_chair,use_the_computer
get_water:1      go_kitchen_sink,go_kitchen_shelf,get_glass,get_water,drink_water
...
```

The community descriptor may include a numeric identifier after a colon; only the part before the colon is used as the community name.

The class exposes two public methods. `loadActivityToCommunity()` returns a `Map<String, String>` that maps each activity label to its community name. This map is used in the enrichment phase. The second method, `loadCommunities()`, returns a `Map<String, List<String>>` that maps each community name to the ordered list of its member activities. This second map is used by the `CepDetector` to instantiate the patterns, since the size and ordering of the list influence the pattern construction for all eight strategies.

10 Detection Phase

10.1 Keyed Stream and Watermarks

Before CEP can be applied, the enriched stream must be converted into a `KeyedStream`. In the smart-home scenario the key is the constant string `"house_01"`, because the dataset contains a single resident and there is a single trace.

Watermarks are assigned using `WatermarkStrategy.forMonotonousTimestamps()`, which is appropriate because the input CSV is already sorted by timestamp. This strategy advances the watermark to the timestamp of the last observed event and does not buffer any events, allowing the job to complete as soon as the file is fully consumed without waiting for an additional out-of-order tolerance period.

10.2 Pattern Application

The `CepDetector` class contains two public static methods. The lower-level `detect()` method accepts a pre-built `KeyedStream` and a pre-built `Pattern` object, applies `CEP.pattern()` to obtain a `PatternStream`, and attaches a `PatternProcessFunction` that collects the matched events, sorts them chronologically, and emits a `DetectedRoutine`.

The higher-level `detectSimple()` method is the entry point used by `Main`: it receives the raw enriched stream, the community name, the chosen `Strategy` enum value, the list of community activities, the percentage parameter, and the maximum time window. It constructs the keyed stream, delegates pattern construction to `RoutinePatternFactory.build()`, and forwards both to `detect()`.

Inside the `PatternProcessFunction`, all events collected across the named pattern steps are merged into a single list, sorted by `Instant` timestamp, and used to compute the start and end instants of the match. The `caseId` is fixed to `"house_01"`. The resulting `DetectedRoutine` is emitted to the output stream.

10.3 RoutinePatternFactory

The `RoutinePatternFactory` class is the component responsible for translating a `Strategy` value into a concrete FlinkCEP `Pattern`. All eight strategies are implemented in the single static method `build()`, which dispatches on the strategy via a Java `switch` expression. In all cases, `AfterMatchSkipStrategy.skipPastLastEvent()` is applied to prevent overlapping matches.

The two implementations — smart-home and taxi — share the same strategy identifiers but differ in how several strategies are realised, reflecting the structural differences between the two domains. The differences are described in detail in the per-strategy sections below.

11 Strategy Implementation Detail

11.1 How the Percentage Strategies Work

Strategies E, F, and G introduce a parameter $p \in (0, 1]$ that relaxes the event count requirement. The intuition is that requiring all N events of a community to appear in one window is often too strict: a

routine can be recognised even when some of its associated sensors are not activated in a particular instance.

The two implementations compute the reduced count differently. The smart-home implementation uses `Math.ceil`:

```
int number = (int) Math.ceil(activities.size() * percent);
```

The taxi implementation uses a cast to `int`, which truncates towards zero and is equivalent to `Math.floor`:

```
int number = (int) (activities.size() * percent);
```

The choice of rounding function has a concrete effect on small communities. For a community of size $N = 5$ with $p = 0.8$: `Math.ceil(4.0) = 4` and `(int)(4.0) = 4` — identical. For $N = 4$ with $p = 0.8$: `Math.ceil(3.2) = 4` whereas `(int)(3.2) = 3`. The smart-home implementation therefore requires a strictly higher count for communities whose size is not a multiple of $1/p$, making it marginally more demanding.

For the taxi scenario, with Manhattan having $N = 69$ zones and $p = 0.8$: both formulas yield $n = 55$. With $p = 0.1$: $n = 7$ (`ceil`) or $n = 6$ (`floor`). The difference is negligible at this scale.

11.2 Strategy A

The two implementations of Strategy A differ substantially in what they check.

Smart-home. Strategy A is implemented as a dynamic chain of `followedByAny()` steps, one per activity in the community list. Each step verifies two conditions jointly: community membership and exact activity identity.

```
Pattern.<EnrichedEvent>begin("step_0", skipStrategy)
    .where(e -> e.community.equals(communityName)
        && e.activity.equals(activities.get(0)));

for (int i = 1; i < activities.size(); i++) {
    pattern = pattern.followedByAny("step_" + idx)
        .where(e -> e.community.equals(communityName)
            && e.activity.equals(activities.get(idx)));
}
```

`followedByAny()` applies non-deterministic relaxed contiguity: events from other communities may appear between two consecutive matched steps, and the NFA explores all possible combinations of matching events. The order of the steps follows the community activity list, which is determined by Infomap's internal node numbering and has no relationship to the natural execution order of the routine. No time window is applied.

Taxi. Strategy A checks only community membership and uses a single looping quantifier `.times(N)` without enforcing any specific activity ordering.

```
Pattern.<EnrichedEvent>begin("start", SKIP)
    .where(e -> communityName.equals(e.community))
    .times(activities.size());
```

Any N events whose `community` field matches the target community name satisfy the pattern, regardless of which specific activities they represent or in what order they appear.

The structural difference between the two implementations reflects the different nature of the two domains. In the smart-home scenario, the activity identity carries semantic meaning: the system distinguishes between `get_glass` and `go_kitchen_shelf` even within the same community. In the taxi scenario, the relevant information is only that a pickup occurred within a given borough; the specific zone does not need to be tracked at the pattern level.

11.3 Strategy B

Strategy B is implemented as a linear chain of `followedBy()` steps in both scenarios, but the condition checked at each step differs.

Smart-home. Each step checks both community membership and exact activity identity, using relaxed contiguity (`followedBy()`):

```
Pattern.<EnrichedEvent>begin("step_0", skipStrategy)
    .where(e -> e.community.equals(communityName)
        && e.activity.equals(activities.get(0)));

for (int i = 1; i < activities.size(); i++) {
    pattern = pattern.followedBy("step_" + idx)
        .where(e -> e.community.equals(communityName)
            && e.activity.equals(activities.get(idx)));
}
```

Taxi. Each step checks only the exact activity name (i.e., the zone identifier), without verifying community membership:

```
Pattern.<EnrichedEvent>begin("step_0", skipStrategy)
    .where(e -> e.activity.equals(activities.get(0)));

for (int i = 1; i < activities.size(); i++) {
    pattern = pattern.followedBy("step_" + idx)
        .where(e -> e.activity.equals(activities.get(idx)));
}
```

In both implementations, the order is strict: `step_i` must be matched before `step_i+1` in the stream, following the order of the community activity list. No time window is applied in either case.

The critical limitation of Strategy B is identical in both domains: the community activity list is not ordered by natural execution sequence. In the smart-home scenario, activities are ordered by Infomap's internal node numbering; in the taxi scenario, zones are ordered by `LocationID`, which follows an alphanumeric convention with no operational or geographic meaning. In both cases the probability of events arriving in exactly the list order is negligible, and Strategy B produces almost no matches in practice.

11.4 Strategy C

Strategy C is identical in both implementations. It checks only community membership and uses `.times(N).allowCombinations()`, which instructs the NFA to explore all possible subsets of N matching events and may produce multiple overlapping matches for the same input subsequence.

```
Pattern.<EnrichedEvent>begin("start", SKIP)
    .where(e -> communityName.equals(e.community))
    .times(activities.size())
    .allowCombinations();
```

This increases recall at the cost of potential duplicate detections. In practice, the difference from Strategy A is only observable when the same community events repeat frequently within the time window, which does not occur at the scales of either dataset tested.

11.5 Strategy D

Strategy D is identical in both implementations. It checks only community membership and adds `.consecutive()` to the exact-count quantifier, requiring that no events from other communities appear between the N matched events.

```
Pattern.<EnrichedEvent>begin("start", SKIP)
    .where(e -> communityName.equals(e.community))
    .times(activities.size())
    .consecutive();
```

If a non-community event arrives while the NFA is collecting the run, the partial match is discarded and the NFA resets. This is the strictest exact-count strategy and is most effective for communities whose events cluster tightly in time with no interleaving activations from other communities.

11.6 Strategies E and F

Strategies E and F relax the count requirement from N to n , computed as described in Section 11.1. The two implementations differ in contiguity behaviour and in the use of `timesOrMore` vs. `times`.

Smart-home. Both strategies use `.timesOrMore(n)`, which requires *at least* n matching events rather than exactly n . Strategy E applies no additional modifier (relaxed contiguity, one match per NFA state), while Strategy F adds `.allowCombinations()` (non-deterministic relaxed contiguity, multiple overlapping matches).

```
// Strategy E
Pattern.<EnrichedEvent>begin("start", skipStrategy)
    .where(e -> communityName.equals(e.community))
    .timesOrMore(number);
```

```
// Strategy F
Pattern.<EnrichedEvent>begin("start", skipStrategy)
    .where(e -> communityName.equals(e.community))
    .timesOrMore(number)
    .allowCombinations();
```

Taxi. Both strategies use `.times(n)`, requiring exactly n matching events. Strategy E adds `.allowCombinations()`; Strategy F applies no additional modifier.

```
// Strategy E (taxi)
Pattern.<EnrichedEvent>begin("start", SKIP)
    .where(e -> communityName.equals(e.community))
    .times(number)
    .allowCombinations();
```

```
// Strategy F (taxi)
Pattern.<EnrichedEvent>begin("start", SKIP)
    .where(e -> communityName.equals(e.community))
    .times(number);
```

The use of `timesOrMore` in the smart-home implementation means that a match can include more than n events if additional community events are present in the window. This makes the smart-home E and F more permissive than their taxi counterparts, which require exactly n events.

11.7 Strategy G

Strategy G combines the percentage relaxation of E and F with strict contiguity, but the two implementations realise this in fundamentally different ways.

Smart-home. Strategy G is implemented as a chain of `followedBy()` steps of length n , checking both community membership and exact activity identity at each step, mirroring the structure of Strategy B but limited to the first n activities in the list.

```
Pattern.<EnrichedEvent>begin("step_0", skipStrategy)
    .where(e -> e.community.equals(communityName)
        && e.activity.equals(activities.get(0)));

for (int i = 1; i < number; i++) {
    pattern = pattern.followedBy("step_" + idx)
        .where(e -> e.community.equals(communityName)
            && e.activity.equals(activities.get(idx)));
}
```

Events from other communities may appear between matched steps (`followedBy()` uses relaxed contiguity), but the matched activities must correspond to the first n entries in the community list in order. As with Strategy B, this ordering is determined by Infomap's internal node numbering and does not reflect the natural execution sequence of the routine.

Taxi. Strategy G uses `.times(n).consecutive()`, requiring exactly n strictly consecutive community events with no interleaving events from other communities.

```
Pattern.<EnrichedEvent>begin("start", SKIP)
    .where(e -> communityName.equals(e.community))
    .times(number)
    .consecutive();
```

The structural difference is significant. The taxi implementation enforces strict physical contiguity in the stream; the smart-home implementation enforces a specific activity ordering but tolerates interleaving events between matched steps.

11.8 Strategy H

Strategy H is the most permissive strategy, but its implementation differs substantially between the two scenarios, reflecting a fundamental difference in what the two systems are trying to detect.

Smart-home. Strategy H detects a temporal *gap* between two events of the same community, that is, a situation where no community event is observed for at least `tMax` time, followed by a new community event. It uses `notFollowedBy()` to express the absence of a community event within the window, and `followedBy()` to require a subsequent community event after the gap.

```
Pattern.<EnrichedEvent>begin("before_gap", skipStrategy)
    .where(e -> communityName.equals(e.community))
    .notFollowedBy("during_gap")
    .where(e -> communityName.equals(e.community))
```

```
.within(tMax)
.followedBy("after_gap")
.where(e -> communityName.equals(e.community));
```

This pattern fires when a community event occurs, followed by a silence of at least `tMax` with no further community events, followed by another community event. The semantic is therefore: two distinct activations of the same community that are separated by a gap of at least `tMax`. This is conceptually different from all other strategies: it detects temporal separation rather than co-occurrence.

Taxi. Strategy H uses `.timesOrMore(2).consecutive().within(tMax)`, requiring at least two strictly consecutive community events within the time window.

```
Pattern.<EnrichedEvent>begin("start", SKIP)
  .where(e -> communityName.equals(e.community))
  .timesOrMore(2)
  .consecutive()
  .within(tMax);
```

Unlike the smart-home implementation, the taxi Strategy H fires on compact runs of community events, not on gaps between them. The minimum of two events prevents the pattern from firing on every isolated pickup, providing a minimal co-occurrence requirement that is still easily satisfied in high-traffic boroughs such as Manhattan.

The divergence between the two implementations of Strategy H illustrates a broader point about the architecture: the eight strategy identifiers define a conceptual space of detection policies, but the concrete realisation of each policy must be adapted to the structure and semantics of the target domain. The community file is not the only domain-specific artefact: the pattern factory itself may require domain-specific tuning.

Chapter 5

Comparative Analysis

This chapter discusses the conceptual analogy between the two case studies and summarises the results obtained by running the system on both datasets.

12 The Analogy Between the Two Domains

Despite operating in entirely different domains, the two systems are built on the same conceptual foundation.

The full implementation is available at [this Google Drive folder](#).

In the smart-home domain, the *Use Computer* routine shows the same behaviour: on different days the resident may turn the computer on before sitting down, or after. The sequence of IoT events differs across instances, yet they all belong to the same community because they involve the same correlated sensors.

In the taxi domain, a vendor operating in Manhattan picks up passengers from different zones on different days. The specific sequence of zones differs, yet all these operational patterns belong to the Manhattan community because the vendor's territory defines the high-level routing behaviour. Two routes that both stay within Manhattan belong to the same community, even if they visit entirely different specific zones.

This analogy demonstrates that the two-phase architecture is a general-purpose framework applicable to any domain where a stream of events can be meaningfully grouped into communities. However, the community file is not the only domain-specific artefact: as shown in Chapter 4, the `RoutinePatternFactory` itself requires domain-specific adaptation. Strategy A uses a chain of `followedByAny()` steps with a joint community-and-activity condition in the smart-home implementation, and a single `.times(N)` quantifier checking only community membership in the taxi implementation. Strategy H detects a temporal gap between two community events in the smart-home scenario, and a consecutive run of at least two community events in the taxi scenario. Strategies E, F, and G differ in their use of `timesOrMore` versus `times` and in the rounding function applied to the percentage count. The pipeline structure, the data model, and the CEP engine are fully reusable across domains; the pattern factory requires tuning whenever the structural properties of the target domain differ significantly from those of the original scenario.

13 Smart-Home Results

This section presents the results obtained by running the detection pipeline on the smart-home IoT dataset, which records sensor activations of a single resident (**house_01**) over a period of approximately three weeks in March–April 2020. All eight strategies were executed. The percentage parameter p was set to 0.8 for Strategies E, F, and G.

13.1 Detection Counts

Table 3 reports the total number of matches detected per strategy and community. A dash indicates zero matches. Entries marked with † denote anomalous matches whose duration spans a large fraction of the entire dataset and are discussed separately in Section 13.8.

Table 3. Total matches detected per strategy and community (smart-home dataset, $p = 0.8$).

Community	A	B	C	D	E	F	G	H
change_clothes	–	–	23	16	28	28	1 [†]	117
do_exercise	3	3	3	3	3	3	3	–
get_clothes	12	12	13	4	13	13	12	16
get_water	–	1 [†]	75	5	86	86	1 [†]	257
go_bed	1 [†]	1 [†]	18	–	18	18	1 [†]	21
go_chair	181	181	191	182	191	191	181	186
go_computer	25	25	39	23	39	39	25	70
go_dining_table	–	–	24	–	24	24	–	38
go_windows	1 [†]	1 [†]	2 [†]	–	3	3	2	6
interact_with_man	20	20	22	20	22	22	20	20
wash_hands	–	–	28	–	35	35	–	45
wc_do	22	22	60	19	76	76	22	132
Total	264	265	476	272	518	518	267	908

13.2 Strategies A and B

Strategies A and B produce almost identical match counts across all communities. Both use a chain of steps that check community membership and exact activity identity jointly: Strategy A uses `followedByAny()` while Strategy B uses `followedBy()`. The practical difference is negligible for most communities, because the activity sequences in this dataset are not rich enough in repeated events within a single trace to produce diverging results under the two contiguity modes.

The only difference between A and B is that Strategy B finds one match for **get_water** while Strategy A finds none. This match is anomalous: it spans from 16 March to 28 March, approximately 12 days, and collects activities across multiple unrelated sessions. The match arises because Strategy B uses `followedBy()` without a time window, so the NFA waits indefinitely for the next activity in the community list order. Since the list order is determined by Infomap’s internal node numbering rather than by the natural execution sequence of the routine, the probability of events arriving in exactly that order within a compact time interval is negligible. When no time window constrains the search, the NFA eventually finds the required sequence by spanning the entire dataset. The same mechanism produces the anomalous single matches for **go_bed** and **go_windows** in both strategies.

Strategy A and B are therefore not useful for detecting compact, meaningful routine occurrences in this dataset. They are included for completeness and as a baseline against which the other strategies can be compared.

13.3 Strategy C

Strategy C uses `followedByAny()` combined with `allowCombinations()`, which instructs the NFA to explore all possible subsets of matching events and produces substantially more matches than Strategy A. The results show a clear improvement in recall across all communities.

Communities that were entirely undetected by A and B now produce meaningful match counts: `change_clothes` yields 23 matches, `go_dining_table` 24, and `wash_hands` 28. The `get_water` community, which collapsed to a single anomalous match under Strategy B, now produces 75 matches. `go_bed` jumps from 1 anomalous match to 18 genuine ones, and `wc_do` from 22 to 60.

The improvement is explained by the `allowCombinations()` modifier: rather than committing to the first valid combination of events, the NFA explores all possible subsets of matching events within the window, firing a match for each distinct combination. This is particularly effective for communities whose events repeat frequently within the same time window, such as `get_water` and `wc_do`, where the resident may activate the same sensors multiple times in a single session.

The two anomalous matches for `go_windows` persist in Strategy C, though their count increases to 2. This reflects the fact that `go_windows` is a rare routine with only a handful of occurrences in the dataset, and `allowCombinations()` generates slightly more combinations from the same sparse event sequence.

13.4 Strategy D

Strategy D uses `.times(activities.size()).consecutive()`, requiring at least n strictly consecutive community events within the time window, with no events from other communities in between. It is the strictest strategy in terms of contiguity, and its results reveal clearly which communities tend to be executed in uninterrupted bursts and which are frequently interleaved with other activities.

The most striking result is that several communities that Strategy C detects in abundance are severely reduced or eliminated by Strategy D. `wash_hands` drops to zero, `go_bed` to zero, `go_dining_table` to zero, and `go_windows` to zero. `get_clothes` drops from 13 to 4, and `wc_do` from 60 to 19. These reductions reflect the fact that the resident's behaviour is frequently interleaved: even during a well-defined routine such as washing hands or going to bed, sensor activations from other communities appear between consecutive steps, breaking the strict contiguity requirement.

`go_chair` (182 matches) and `interact_with_man` (20 matches) are essentially unaffected by the strict contiguity requirement. Both communities have only two activities, and consecutive pairs of same-community events are common even in a heavily interleaved stream.

`change_clothes` behaves unexpectedly well under D, producing 16 matches despite being a large community ($N = 11$). Examining the matched sequences reveals why: the matches consist of long consecutive runs of wardrobe-related activities such as `go_wardrobe`, `change_clothes`, `dress_up_outdoor`, and `go_workplace`. When the resident is in the middle of getting dressed or undressed, they activate multiple sensors from the same community in rapid succession without interruption from other routines. This is a genuine behavioural pattern: the change clothes routine is one of the few routines in this dataset that tends to be executed compactly.

`get_water` produces only 5 matches under D, and examining the matched sequences shows runs of kitchen-related activations that happen to be uninterrupted. These are the rare instances when the resident went to the kitchen, prepared a drink, and completed the sequence without any intervening activity from another community.

Overall, Strategy D is the most informative strategy for understanding the internal structure of each community's execution pattern. Communities with high D counts are executed compactly; communities with low or zero D counts are executed in an interleaved manner. This information could be used to choose the most appropriate strategy for each community individually, rather than applying the same strategy to all communities.

13.5 Strategies E and F

Strategies E and F produce identical results across all communities. Both use `timesOrMore(n)` with $n = \lceil N \times 0.8 \rceil$, requiring at least n matching events. Strategy F adds `allowCombinations()`, but this modifier generates no additional matches at the community sizes present in this dataset. The two strategies are therefore empirically equivalent for this scenario.

Compared to Strategy C, E and F produce a further increase in match counts across most communities. `get_water` grows from 75 to 86, `wc_do` from 60 to 76, and `wash_hands` from 28 to 35. `change_clothes` grows from 23 to 28. The increase is attributable to the `timesOrMore` quantifier: unlike `times(n)`, which requires exactly n events, `timesOrMore(n)` accepts any number of events greater than or equal to n . When more than n community events are present in the window, the NFA greedily captures them all in a single match rather than stopping at n .

The `go_windows` anomaly resolves under E and F: instead of two matches spanning multiple days, three compact matches are detected, each covering a short time interval. This suggests that the `timesOrMore` quantifier, combined with the absence of strict ordering, finds genuine short-duration occurrences that the ordered chain of Strategy C misses.

13.6 Strategy G

Strategy G applies the same activity-ordered chain structure as Strategy B, but limits the chain length to $n = \lceil N \times 0.8 \rceil$ instead of N . The results reveal a striking pattern: for communities with small N or with activities that tend to appear in list order, G performs comparably to A and B. For larger communities with activities that rarely appear in list order, G collapses to single anomalous matches that span large portions of the dataset.

`get_water` ($N = 8, n = 7$) produces a single match spanning from 16 March to 26 March. `change_clothes` ($N = 11, n = 9$) produces a single match spanning from 16 March to 26 March. `go_bed` produces a single match spanning the entire dataset. These are the same anomalous long-duration matches observed in Strategy B, now with a slightly shorter required chain but with the same fundamental problem: the activity list order is determined by Infomap’s internal node numbering and does not reflect the natural execution sequence of the routine.

Communities with $N = 2$ (`go_chair`, `interact_with_man`, `do_exercise`) are unaffected, producing the same counts as A and B. For these communities, $n = \lceil 2 \times 0.8 \rceil = 2$, which is the full community size, and the two activities appear frequently in the correct order because the list has only two elements and the NFA does not need to wait long to find a matching pair.

`get_clothes` ($N = 3, n = 3$) and `wc_do` ($N = 5, n = 4$) behave well under G, producing 12 and 22 matches respectively, identical to Strategy A. This suggests that for these communities the first n activities in the Infomap list happen to correspond to activities that do occur in that order frequently enough in the dataset. `go_windows` produces 2 compact matches under G, an improvement over the anomalous single match of Strategy B, because the reduced chain length of 4 instead of 5 is more easily satisfied in a short window.

Overall, Strategy G is the weakest performer among the non-trivial strategies. The combination of activity-identity checking and list-order enforcement, inherited from Strategy B, makes it highly sensitive to the quality of the activity list ordering. In this dataset, where the ordering is an algorithmic artefact, Strategy G provides no practical advantage over A or B and introduces anomalous long-duration matches for several communities.

13.7 Strategy H

Strategy H uses `timesOrMore(2).consecutive().within(tMax)`, requiring at least two strictly consecutive community events within the time window. It is the most permissive strategy in terms of count requirement, but it enforces physical contiguity: the two matched events must appear one immediately after the other in the stream with no events from other communities in between.

Strategy H produces the highest match counts of all strategies for most communities: `get_water` reaches 257 matches, `change_clothes` 117, `wc_do` 132, `go_computer` 70, and `wash_hands` 45. These counts are substantially higher than those of E and F, which already produce the second-highest counts. The reason is twofold. First, the minimum count of 2 is easily satisfied: any pair of consecutive community events produces a match. Second, the `timesOrMore` quantifier is greedy, so the NFA captures the longest possible consecutive run before emitting a match and then resets, allowing multiple short matches to be extracted from a single dense session of community activity.

Examining the matched event sequences confirms this interpretation. The first `get_water` match covers only `go_kitchen_shelf` and `get_glass`, with a duration of one minute. The second covers `go_kitchen_sink` and `get_water`, with a duration of zero minutes. These are not routine detections in any meaningful sense: they are pairs of adjacent sensor activations that happen to belong to the same community. The system is detecting co-location, not behaviour.

Notably, `do_exercise` produces zero matches under Strategy H. This community has only two activities (`go_exercise_place` and `do_exercise`), and the two events apparently never appear strictly consecutively in the stream without an intervening event from another community. This is a concrete illustration of how strict contiguity can eliminate matches even for small communities when the resident’s behaviour is interleaved at a fine granularity.

`interact_with_man` produces exactly 20 matches under H, identical to Strategy A. This suggests that the two activities of this community (`go_entrance` and `interact_with_man`) always appear strictly consecutively, so the contiguity constraint of H is never a limiting factor.

Overall, Strategy H is the upper bound on detectability for this dataset. It reveals how many pairs of consecutive community events exist in the stream, but it does not distinguish between a genuine routine occurrence and two accidentally adjacent sensor activations. It is most useful as a diagnostic tool to understand the density of community activity in the data, rather than as a practical routine detection strategy.

13.8 Anomalous Long-Duration Matches

Several strategies produce matches whose duration spans a large fraction of the dataset. Table 4 summarises the observed anomalous matches.

Table 4. Anomalous long-duration matches in the smart-home dataset.

Community	Strategy	Start	End
go_bed	A, B, G	2020-03-16	2020-04-06
get_water	B	2020-03-16	2020-03-28
get_water	G	2020-03-16	2020-03-26
go_windows	A, B	2020-03-21	2020-03-22
go_windows	C	2020-03-22	2020-04-02
change_clothes	G	2020-03-16	2020-03-26

All anomalous matches share the same root cause: the NFA is searching for activities in a specific order derived from the community list, and that order does not match the natural execution sequence of the routine. Without a binding time window (Strategies A and B have no `within()` constraint), or with a chain structure that allows indefinite waiting between steps (Strategy G uses `followedBy()` without `within()`), the NFA accumulates partial states indefinitely until it finds the required sequence, even if the matching events are separated by days of unrelated activity.

These matches are statistically meaningless as routine detections. They do not represent a resident performing a routine; they represent the NFA finding the required activities scattered across the entire observation period. They should be filtered out in any practical deployment, for example by enforcing a maximum match duration as a post-processing step.

14 NYC Taxi Results

This section presents the results obtained by running the detection pipeline on the NYC Yellow Taxi dataset (January 2021, Vendor 1 only). Five strategies were executed: A, D, F, G, and H. Strategies B, C, and E were excluded: B because the zone list within each borough community is ordered by `LocationID` and therefore produces no meaningful matches in practice (as discussed in Section 11.3); C and E because both rely on `allowCombinations()`, which causes the NFA to explore an exponential number of subsets for communities of size 43–69 zones, making these strategies computationally intractable within the available time and memory bounds.

The percentage parameter p was set to 0.8 for Strategies F and G. The dataset covers 31 active vendor-days for Vendor 1 in January 2021, yielding 31 unique cases. The five borough communities have the following sizes: Manhattan 69 zones, Queens 69, Brooklyn 61, Bronx 43, Staten Island 20.

14.1 Detection Counts

Table 5 reports the total number of matches detected per strategy and borough. A dash indicates zero matches. Table 6 reports the number of unique vendor-day cases that produced at least one match.

Table 5. Total matches detected per strategy and borough (NYC Taxi, Vendor 1, January 2021, $p = 0.8$).

Borough	A	D	F	G	H
Bronx	11	–	19	–	14
Brooklyn	30	–	44	–	28
Manhattan	4,810	683	6,011	1,195	157,611
Queens	261	–	332	–	1,599
Staten Island	–	–	–	–	–
Total	5,112	683	6,406	1,195	159,252

Table 6. Number of unique vendor-day cases covered per strategy and borough (out of 31 active days).

Borough	A	D	F	G	H
Bronx	11	–	18	–	13
Brooklyn	24	–	31	–	18
Manhattan	31	31	31	31	31
Queens	31	–	31	–	31
Staten Island	–	–	–	–	–

14.2 The Role of Community Size

The single most important factor determining strategy effectiveness in the taxi scenario is community size. With Manhattan and Queens each containing 69 zones, Brooklyn 61, and the Bronx 43, even the percentage-based strategies require a large number of daily pickups to fire. Strategy F requires $\lceil 69 \times 0.8 \rceil = 56$ pickups in Manhattan or Queens, $\lceil 61 \times 0.8 \rceil = 49$ in Brooklyn, and $\lceil 43 \times 0.8 \rceil = 35$ in the Bronx. Strategy D requires all 69, 61, and 43 respectively, in strictly consecutive order.

These thresholds are structurally different from the smart-home scenario, where communities have at most 11 members and the same percentage relaxation yields thresholds of 3 to 9 events. In the taxi domain, the community definition at borough granularity is simply too coarse for most strategies to fire reliably outside the densest borough.

14.3 Strategies A and F

Strategies A and F produce moderate match counts across the boroughs where activity was recorded. Strategy F consistently outperforms A because the reduced count n lowers the threshold while maintaining the same permissive contiguity: non-matching events from other communities may appear between any two matched pickups within the 24-hour window.

Manhattan and Queens achieve full case coverage under both strategies: all 31 active vendor-days produce at least one match. This confirms that Vendor 1 accumulates enough pickups in both boroughs on every working day to satisfy the respective thresholds within 24 hours.

Brooklyn shows a notable gap under Strategy A: only 24 of 31 cases produce a match. Strategy F recovers the remaining 7 cases, achieving full coverage of 31, demonstrating that the exact-count requirement of A is the binding constraint on the days where A fails. On those days, Vendor 1 performs between 49 and 60 Brooklyn pickups — enough for F but not for A’s requirement of 61.

The Bronx is the most affected borough. Strategy A covers only 11 of 31 cases, and Strategy F covers 18. On the remaining days, the vendor’s Bronx activity is simply insufficient to meet either threshold. The Bronx is a lower-traffic borough visited intermittently between Manhattan sessions, and on many days the vendor does not accumulate enough Bronx pickups in a single 24-hour window for the pattern to fire.

Staten Island produces zero matches under both strategies. The borough has 20 zones, requiring $\lceil 20 \times 0.8 \rceil = 16$ pickups for F and all 20 for A. Vendor 1 does not operate in Staten Island with sufficient regularity or volume for either threshold to be met on any day in the dataset.

14.4 Strategies D and G

Strategies D and G produce matches exclusively on Manhattan. Strategy D requires all 69 Manhattan zones to appear strictly consecutively in the stream, with no pickups from other boroughs in between. Strategy G relaxes the count to 56 but retains strict contiguity. Despite the demanding conditions, both strategies fire on every active day in Manhattan: D produces 683 matches across all 31 cases, and G produces 1,195 matches across all 31 cases.

The high match counts on Manhattan, despite the strict contiguity requirement, reflect the extreme density of Manhattan pickups. Vendor 1 makes so many consecutive Manhattan pickups in rapid succession that runs of 56 or even 69 uninterrupted borough pickups are not rare — they occur repeatedly throughout a single working shift. This is fundamentally different from the smart-home scenario, where strict contiguity is a strong filter that eliminates matches for interleaved routines.

Outside Manhattan, both D and G produce zero matches. For Brooklyn, Queens, and the Bronx, the vendor’s daily pickup pattern involves frequent trips across borough boundaries, so runs of 49, 56, or more consecutive same-borough pickups never occur. The strict contiguity requirement is therefore not a tuning parameter for these boroughs — it is an absolute barrier.

The higher match count of G compared to D (1,195 versus 683) reflects the lower threshold: requiring 56 consecutive Manhattan pickups instead of 69 allows more runs to qualify, and since `skipPastLastEvent()` resets the NFA after each match, a lower threshold allows more non-overlapping matches to be extracted from the same dense stream.

14.5 Strategy H

Strategy H uses `timesOrMore(2).consecutive().within(tMax)`, requiring at least two strictly consecutive community pickups within the time window. It produces by far the highest match counts of all strategies: 157,611 matches on Manhattan alone, 1,599 on Queens, 28 on Brooklyn, and 14 on the Bronx.

The extreme count on Manhattan is a direct consequence of the low threshold: any two consecutive Manhattan pickups satisfy the pattern. Given the density of Manhattan activity, the NFA fires on nearly every pair of adjacent pickups, resets immediately via `skipPastLastEvent()`, and fires again. The average

match duration for Manhattan under H is approximately zero, confirming that most matches consist of exactly two adjacent pickups with near-identical timestamps.

The discrepancy between Manhattan (157,611 matches, 31 cases) and Brooklyn (28 matches, 18 cases) illustrates the density gap between the boroughs. Even the minimum requirement of two consecutive same-borough pickups is rarely met in Brooklyn, because the vendor frequently alternates between Brooklyn and Manhattan between consecutive trips. For the Bronx, 14 matches across 13 cases suggests that consecutive Bronx pickups occur on most days the vendor visits the borough, but only once or twice per day.

Staten Island produces zero matches under H. This means that Vendor 1 never makes two strictly consecutive Staten Island pickups on any day in January 2021 — every Staten Island trip is separated from the next by at least one pickup in another borough.

14.6 Match Duration and Operational Interpretation

The average match duration differs substantially across strategies and boroughs, and these differences carry operational meaning.

For Manhattan, all strategies report very short average durations: Strategy A averages approximately 10 minutes per match, Strategy D approximately 6 minutes, Strategy F approximately 8 minutes, and Strategy G approximately 5 minutes. This reflects the high pickup density: the vendor accumulates the required number of consecutive Manhattan pickups within minutes, after which `skipPastLastEvent()` resets the NFA and the next match begins almost immediately. The result is that the NFA fires hundreds or thousands of times per vendor-day, each match representing a short burst of concentrated Manhattan activity.

For the Bronx and Brooklyn under Strategies A and F, average durations are substantially longer: approximately 12–17 hours for the Bronx and 10–14 hours for Brooklyn. This means that when a match is found, the vendor has accumulated the required pickups intermittently over most of the working day, visiting the borough in short sessions between longer Manhattan runs. These matches represent a genuine operational pattern: Vendor 1 uses the Bronx and Brooklyn as secondary territories visited throughout the day rather than as concentrated operating zones.

Strategy H reports near-zero average duration for all boroughs, because the minimum of two consecutive pickups is always satisfied within seconds or minutes of the first pickup in the borough.

Chapter 6

Limitations and Future Work

This chapter discusses the known limitations of both implementations and outlines the directions in which the system could be extended or improved.

15 Smart Home Limitations

Imperfect community detection. The Infomap algorithm assigns each event to exactly one community. However, human behaviour does not partition cleanly: the same physical action can belong to multiple routines depending on context. Going to the bathroom sink is part of the WC routine, but it also appears in the Eat Cold routine, the Eat Warm routine, and the Prepared Sleep routine. Infomap cannot know this, it sees only co-occurrence frequencies in the historical log and assigns each activity to the community where it fits best statistically. The result is that some events end up in the wrong community from a semantic standpoint. A concrete example is `switch_computer_off`, which was assigned to the `wc_do` community instead of `go_computer`. The Use Computer routine is therefore detected without its natural closing step, and the WC routine is detected with an activity that has nothing to do with it.

The activity order in a community does not reflect human behaviour. The list of activities within each community is ordered by Infomap’s internal node numbering, which is an algorithmic artefact with no relationship to the order in which a person actually performs those activities. A human going to the computer might sit down first and then turn it on, or turn it on first and then sit down. Infomap’s list might say the opposite. Strategies that depend on the list order, specifically Strategy B and Strategy G in the smart-home implementation, will therefore fail to find matches not because the routine is absent, but because the resident does not follow the order that the algorithm happened to produce. This is not a limitation of the resident’s behaviour; it is a limitation of using an unsupervised algorithm’s output as if it carried sequential meaning.

Fixed case identifier. The `caseId` is hardcoded to "house_01" throughout the pipeline. In a multi-resident or multi-home scenario this would need to be parameterised, as all events would incorrectly share the same trace.

No automatic evaluation. The system produces output files but does not compute precision, recall, or F1 scores against the ground truth automatically. Comparison with the segmented event log requires external tools such as PM4Py or Disco.

Static file input. The pipeline reads from a static CSV file. In a real smart-home deployment the source would be a live sensor stream, for example via a Kafka topic, requiring a different source connector and watermark strategy.

16 NYC Taxi Limitations

Borough boundaries are not routing patterns. The most fundamental limitation of the taxi scenario is that the communities are defined geographically rather than behaviourally. A borough is an administrative boundary, not a description of how a taxi driver operates. Two drivers who both work in Manhattan visit entirely different zones, at different times, in different sequences, every single day. What they share is not a routine, it is a territory. Using borough membership as a community definition means that the system can only answer the question “did this driver work in this borough today?”, not “did this driver follow this routing pattern today?”. A genuine community structure for taxi data would need to emerge from the trip sequences themselves, for example by building a Directly-Follows Graph where nodes are zones

and edges represent consecutive pickups, and then applying community detection to discover the corridors that drivers actually tend to follow.

The zone order within a community is meaningless. Even setting aside the borough-level granularity problem, the list of zones within each borough community is ordered by `LocationID`, which is an administrative identifier assigned alphanumerically with no geographic or operational significance. Zone 1 is not next to zone 2. A driver going from Midtown to the Upper East Side is not following a sequence that corresponds to any ordering by `LocationID`. Strategy B, which requires events to appear in exactly the list order, will therefore almost never produce a match, not because the driver is not operating in that borough, but because no driver in the world follows an alphabetical tour of pickup zones. The same issue affects Strategy G in the taxi implementation to a lesser extent: the ordered chain of `followedBy()` steps makes assumptions about sequence that the data simply does not support.

Community sizes are too large for most strategies. With up to 69 zones per borough, exact-count strategies require a driver to visit every zone of a borough in a single day, which is practically impossible. Even with an 80% relaxation, Strategy F requires 56 consecutive Manhattan pickups and Strategy G requires 56 uninterrupted consecutive pickups with no trips to other boroughs in between. These are conditions that only the densest borough can satisfy, and only because Manhattan is so intensely trafficked that a driver naturally accumulates long runs of consecutive Manhattan pickups simply by working there. For smaller boroughs like the Bronx or Staten Island, these thresholds are never reachable. The fundamental problem is that borough-level grouping is too coarse: a finer community structure, such as neighbourhoods within boroughs derived from actual trip transition patterns, would yield communities of a size that the strategies can realistically match.

Strategies C and E are computationally intractable. Both rely on `.allowCombinations()`, which causes the NFA to explore all possible subsets of matching events. The number of candidate subsets grows exponentially with the community size. For Manhattan with 69 zones, or even with the 80% relaxation yielding 56, the NFA state space becomes too large to process within acceptable time and memory bounds. These strategies were therefore excluded from the taxi evaluation entirely.

Only pickup location is used. Every taxi trip has both a pickup zone and a dropoff zone, but only the pickup is used as the event activity. Including the dropoff would allow modelling complete origin-destination pairs rather than just pickup area tendencies, which would enable richer and more operationally meaningful pattern detection.

Output not separated by vendor. When running without the `--vendor` filter, results from both vendors are mixed in the same output files, distinguishable only by the `caseId` field. Adding the vendor identifier to the output filename would make the results easier to analyse separately.

17 Future Work

Live streaming input. Replacing the `FileSource` with a Kafka connector would enable true real-time processing. This would require adjusting the watermark strategy from `forMonotonousTimestamps` to `forBoundedOutOfOrderness` to handle events arriving out of order from different producers.

Automatic evaluation pipeline. Implementing an automated comparison against the ground truth using precision, recall, and F1 score would allow systematic evaluation of each strategy across both datasets without requiring external tools.

Graph-based community detection for the taxi scenario. The most impactful improvement for the taxi scenario would be replacing the borough proxy with communities discovered from the trip data itself. Building a Directly-Follows Graph on the zone-level pickup sequences and applying Infomap would yield communities that reflect actual driver behaviour rather than administrative geography, making the detection strategies both more meaningful and more tractable.

Extension to new domains. The two-phase architecture is domain-agnostic by design. Applying it to new scenarios, such as hospital patient pathways, e-commerce clickstreams, or industrial sensor data, would validate the generality of the approach and potentially reveal new adaptations required for different data characteristics. The taxi scenario already demonstrated one important lesson: the quality of the

community definition matters more than the choice of detection strategy. Any new domain application should begin with a careful analysis of what constitutes a meaningful community in that domain, rather than relying on a convenient proxy.

References

1. Apache Software Foundation: FlinkCEP: Complex event processing for Apache Flink. <https://nightlies.apache.org/flink/flink-docs-stable/docs/libs/cep/> (2024), accessed: 2024
2. Callisto De Donato, M., Fornari, F., Armas-Cervantes, A.: From IoT event logs to human routines via community detection algorithms. In: Proceedings of the BP-Meet-IoT Workshop. Springer (2024)